

NYCU-EE IC LAB - FALL2024

Lab07 Exercise

Design: Convolution with Clock Domain Crossing

Data Preparation

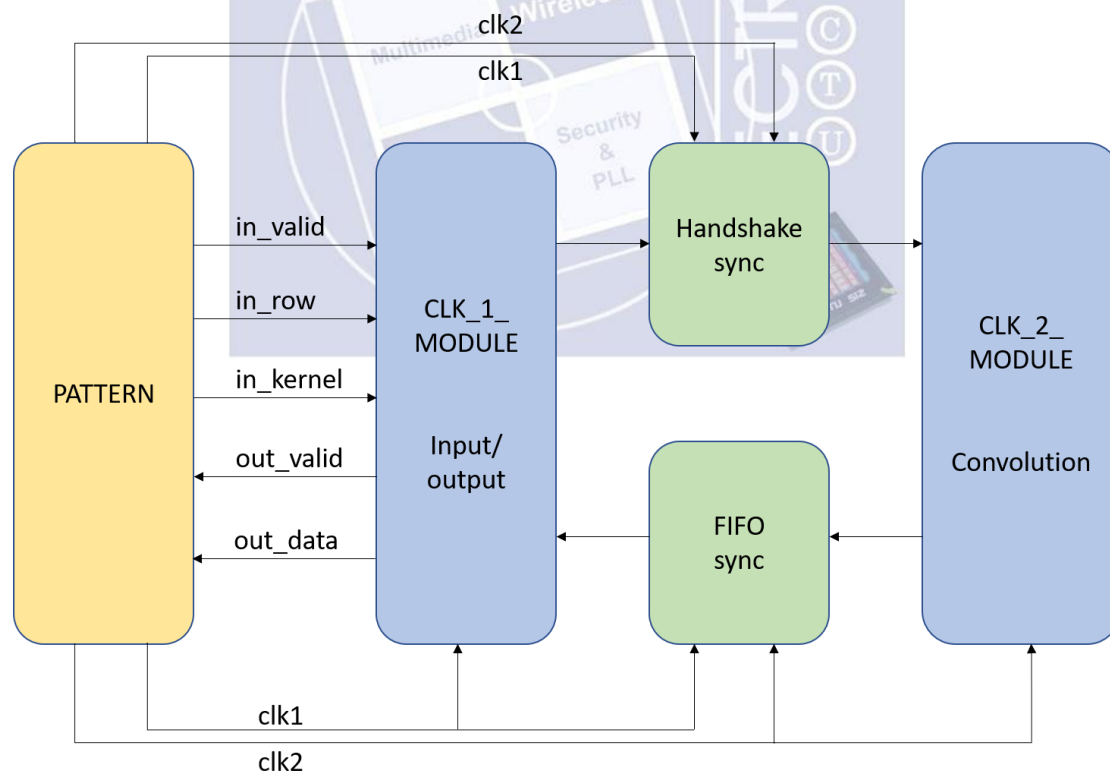
1. Extract Lab directory from TA's directory:

`% tar -xvf ~iclabTA01/Lab07.tar`

Basic Concept

In this lab, you will receive a matrix $M_{6 \times 6}$ and 6 kernel $K_{i,2 \times 2}$ and you should calculate $M_{6 \times 6} * K_{i,2 \times 2} = C_{i,5 \times 5}$. Then, output the matrix C_i element by element, which i is in the range of (0, 5). The detailed structure is described below.

1. The input matrix M and kernel K is given in clk1 domain.
2. Use Handshake synchronizer to transfer the data into clk2 domain.
3. Calculate the result in clk2 domain.
4. Use FIFO synchronizer to transfer the data to clk1 domain.
5. Output the result element by element in the clk1 domain.



Design Description

In this lab, you are asked to implement Convolution.

For input signal in the matrix and the kernel, you will receive the matrix element and the kernel as the following sequence:

$M_0, M_1, M_2, M_3, M_4, M_5$, where $M_j = \{M_{j,5}, M_{j,4}, M_{j,3}, M_{j,2}, M_{j,1}, M_{j,0}\}$.

$K_0, K_1, K_2, K_3, K_4, K_5$, where $K_i = \{K_{i,1,1}, K_{i,1,0}, K_{i,0,1}, K_{i,0,0}\}$.

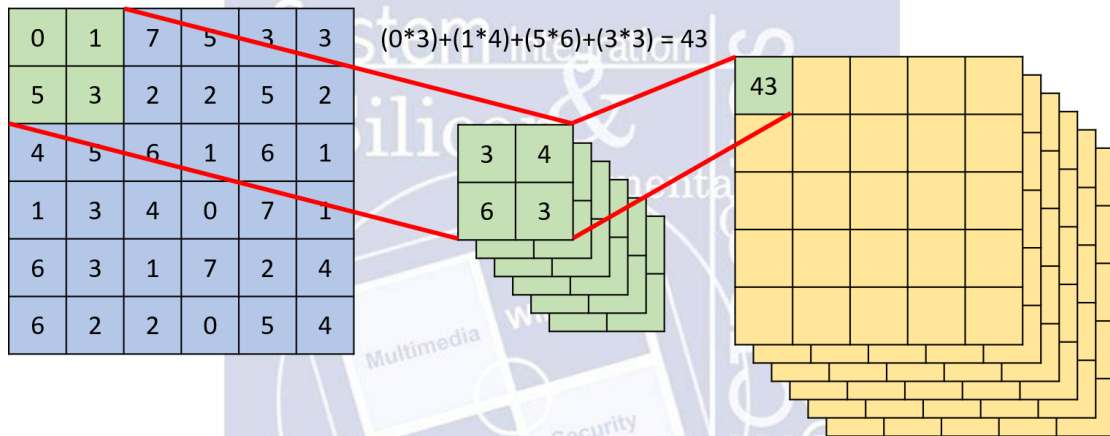
And the convolution equation is:

$$C_{i,j,k} = M_{j,k} * K_{i,0,0} + M_{j,k+1} * K_{i,0,1} + M_{j+1,k} * K_{i,1,0} + M_{j+1,k+1} * K_{i,1,1}$$

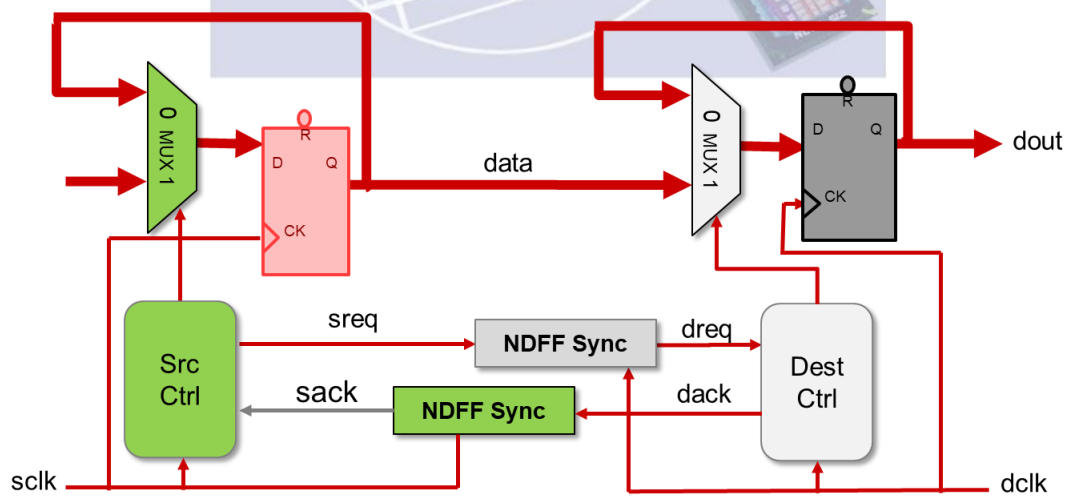
After implementing the Convolution: $M_{6 \times 6} * K_{2 \times 2} = C_{5 \times 5}$. You should output the result as the following sequence:

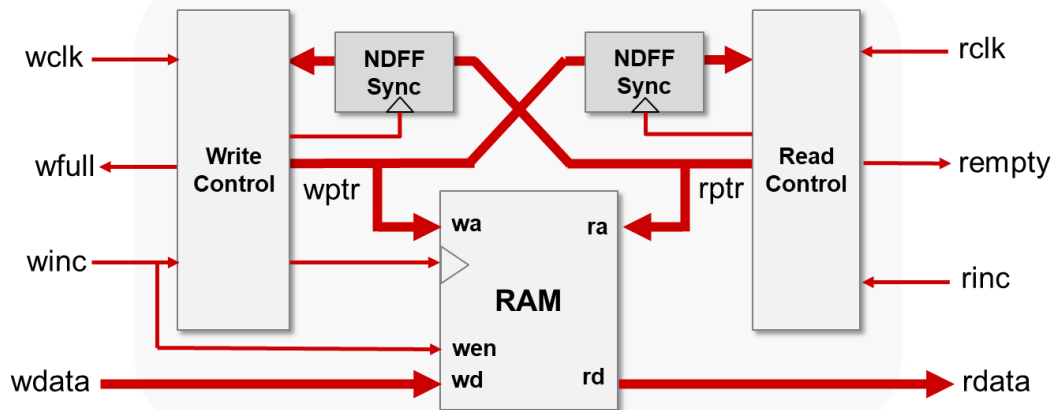
$C_{0,0,0}, C_{0,0,1}, C_{0,0,2}, \dots, C_{5,4,2}, C_{5,4,3}, C_{5,4,4}$

$$\begin{aligned} C_{0,0,0} = & (M_{0,0} * K_{0,0,0}) + (M_{0,1} * K_{0,0,1}) + \\ & (M_{1,0} * K_{0,1,0}) + (M_{1,1} * K_{0,1,1}) \\ & (0*3) + (1*4) + (5*6) + (3*3) = 43 \end{aligned}$$



In this lab, you will also deal with the CDC (cross-chronological domain) problem. The Handshake synchronizer is used to cross $clk1$ to $clk2$. The FIFO synchronizer is used to cross $clk2$ to $clk1$. Handshake and FIFO circuit structures are shown below:





In this lab, you should use JG to verify the CDC design. After running the JG, there should not be an error message in the console, a violation message, and anything in violation.csv. Also, the Paris, Schemes, Convergence, Functional, and Metastability in the CDC Phases section should all be correct. All of the subsection in CDC Configuration should be correct too. (like the figure below)

The figure shows four screenshots from the JG tool interface:

- Console:** Displays warning messages (WDC0241, WDC028, WDC028, WDC021) and a successful check_cdc -waiver -generate command output.
- CDC Phases:** A table showing the status of various CDC phases. All phases are marked as 'Assert' and 'Valid'.
- CDC Configuration:** A table showing the configuration of various signals. All signals are marked as 'Valid'.
- Review Violations:** A window showing 'No rule violation messages'.

Violation Key	Violation Type	Check	Tag	Severity	Rule Value	Design Value	Failure Reason	# Pairs	Source Reset Domain	Destination Re

Inputs

I/O	Signal name	Bits	Description
Input	clk1	1	In 01_RTL : Positive edge trigger clock by clock 1 with 4 different clock period 4.1ns, 7.1ns, 17.1ns, 47.1ns In 03_GATE : Positive edge trigger clock by clock 1 with clock period 47.1ns
Input	clk2	1	Positive edge trigger clock by clock 2 with clock period 10.1 ns
Input	rst_n	1	Asynchronous reset active low reset
Input	in_valid	1	Indicate in_row and in_kernel signals are valid when in_valid is high. This signal is triggered by clk1 for 6 cycles
Input	in_row	18	The unsigned data for the Matrix's row. The matrix's size is 6x6, indicating there are 6 elements in a single row, and each element in the row is 3-bit long. This signal is triggered by clk1 for 6 cycles
Input	in_kernel	12	The unsigned data for the kernel. The kernel's size is 2x2, and each element in the kernel takes 3 bits. This signal is triggered by clk1 for 6 cycles

Outputs

I/O	Signal name	Bits	Description
output	out_valid	1	Should be set to low after reset and not be raised when invalid is high. Should set to high when your out_data is ready. Should be pulled up for 150 cycles, not required to be continuous. This signal is triggered by clk1 .
output	out_data	8	The unsigned result for matrix multiplication. Should be output for 150 cycles, not required to be continuous. This signal is triggered by clk1 .

Specifications

Top module

1. Top module name : CONV_TOP (File name: CONV_TOP.v)
2. Submodule name : CLK_1_MODULE, CLK_2_MODULE
(File name: DESIGN_MODULE.v)
3. Synchronizer name : Handshake_syn, FIFO_syn
(File name: Handshake_syn.v, FIFO_syn.v in synchronizer folder)
4. Synchronizer from TA: NDFF_syn, NDFF_BUS_syn
(File name: NDFF_syn.v, NDFF_BUS_syn.v in synchronizer folder)
5. Dual port SRAM : DUAL_64X8X1BM1 (04_MEM folder)
(Words: 64, Bits: 8, Dual port SRAM)

Reset

6. Use **asynchronous** reset active low architecture.
7. The reset signal (rst_n) would be given only once at the beginning of simulation.
All output signals should be reset after the reset signal is asserted.

Input/Output Signal

8. Input and output data are synchronous to clk1.
9. The out_data should be correct when out_valid is high.
10. The out_data should be reset after your out_valid is pulled down.
11. Output signal out_valid and out_data **should be zero when in_valid is high.**
12. The next input pattern will come in 1~3 clk1 cycles after getting 150 output data.
13. The output should be raised for 150 cycles and is not required to be continuous.

Synthesis and Prime Time

14. Output delay is 0.5 * clk1 Clock Period.
15. Input delay is 0.5 * clk1 Clock Period.
16. The output loading is set to 0.05.
17. **Your design area should not > 2000000.**
18. In the synchronizer you design, please use the NDFF_syn, NDFF_BUS_syn, provided by TA if needed. Prime Time will ONLY **set_annotated_check** to the NDFF_syn module provided by TA.
19. The CONV_TOP.sdc is complete by TA. **DO NOT modify it.** This file is to set the asynchronous clock groups of clk1 and clk2.

```
set_clock_groups -name group1 -asynchronous -group {clk1} -group {clk2}
```

20. After synthesis, check the “CONV_TOP.area” and “CONV_TOP.timing” in the folder “Report”. The area report is valid only when the slack in the end of “CONV_TOP.timing” is **non-negative**.
21. The synthesis result cannot contain any **latch, error, violation, mismatch** (in syn.log).
22. After run Prime Time, the slack in the end of “CONV_TOP_pt.timing” should be also non-negative.
23. The Prime Time result cannot contain any **error, violation** (in syn.log).

Gate level simulation

24. **You can't have timing violation in gate-level simulation.**

Clock period and Latency

25. The design should be able to operate at different output cycles. Please take advantage of the FIFO synchronizers. **TA will demo your design at 4 different clk1 period (4.1ns, 7.1ns, 17.1ns, 47.1ns) in the 01_RTL stage.**
26. **In the 02_SYN and 03_GATE stages, the clk1 period will be fixed at 47.1ns, and latency is calculated based on this.**
27. The latency is from the falling edge of in_valid to the falling edge of out_valid for the last output, including the output cycles!!!!!!
28. Your latency should be **smaller than 5000 cycles in clk1**. The latency cycle is defined as the cycles between the clk1 rising edge of the last input signal and the rising edge of the last output signal.

Dos and Don'ts

29. **Changing top module is prohibited.**
30. Don't use Designware IP.
31. **Calculate the result in CLK_2_MODULE, use CLK_1_MODULE to perform the Matrix Multiplication is prohibited.**
32. **Changing clock period is prohibited. Use the clocks listed above.**
33. TA had generated dual port SRAM for the FIFO synchronizer, and the files are stored in 04_MEM. **Don't modify them.**
34. **You should use dual port SRAM provided by TA to design your FIFO synchronizers to maintain the fairness of area performance.**
35. **Don't modify the parameter “WIDTH” in Handshake_syn.v and FIFO_syn.v**
36. Don't use any wire/reg/submodule/parameter name called ***error***, ***congratulation***, ***pass***, ***latch*** or ***fail*** otherwise you will fail the lab. Note: *** means any char in front of or behind the word. e.g: error_note is forbidden.**
37. Don't write chinese comments or other language comments in the file you turned in. Otherwise, **you will get 5 deduct points.**

Supplement

38. Some pre-defined flags are reserved for you to optimize your design.
39. It's acceptable for the following two warning.

```
Warning-[SDFCOM_CFTC] Cannot find timing check
CONV_TOP_SYN_pt.sdf, 38673
module: QDFFRBS, "instance: TESTBED.I_CONV.u_FIFO_syn_w2r_genblk1_4__u_NDFF_syn.A2_reg"
SDF Warning: Cannot find timing check $hold(posedge CK,posedge RB,...)
```

```
** MEM_Warning: Read and Write the same Address, DO is unknown ( 2185079450 ps) in clock of TESTBED.I_CONV.u_FIFO_syn.
u_dual_sram.ErrorMessage
```

Grading Policy

- ◆ Function correct 70% (01_RTL to 03_GATE, 4 different clk1 cycle times in 01_RTL)
- ◆ Jasper Gold correct 25%
- ◆ Performance: Latency * Area² 5% (Latency is calculated in clk1)
If you didn't pass Function or JG, your score would not include performance.
- ◆ The grade of 2nd demo would be 30% off. And you won't get any points if you fail the function test.
Ex1: Pass function but fail JG in 1st demo. 70%
Ex2: Pass function and JG in 2nd demo. 70% + (25% + performance 5%) * 0.7
Ex3: Pass JG in 1st demo but fail function in 1st demo. 0%
Ex4: Pass JG in 1st demo and fail function in 2nd demo. (70%+25% + performance 5%) * 0.7
- ◆ The latency is from the falling edge of in_valid to the falling edge of out_valid for the last output, including the output cycles.

Note

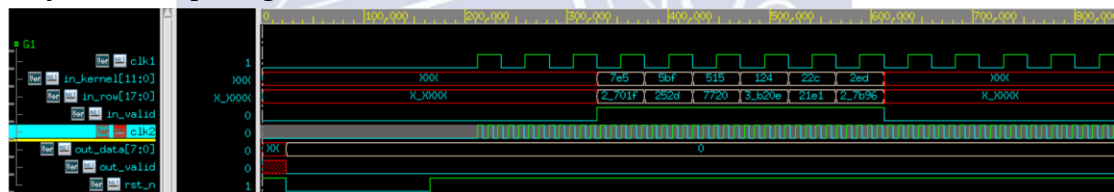
Template folders and reference commands:

1. 01_RTL/ (RTL simulation)
 - I. ./01_run_vcs_rtl
2. 02_SYN/ (Synthesis)
 - I. ./01_run_dc_shell
(Check the design which contains **latch and error** or not in **syn.log**)
 - II. ./02_run_pt
(**set_annotated_check** for the first FF of NDFF synchronizer)
(Check the design's timing in /Report/ CONV_TOP_pt.timing)
3. 03_GATE_SIM/ (GL simulation)
 - I. ./01_run_vcs_gate
(We will only run 47.1ns for clk1 period in 03 Gate Level simulation)
(Check no timing violation)

4. 05_JG/ (CDC verification)
 - I. `./01_run_jg`
5. 09_SUBMIT/ (submit file)
 - I. `./00_tar`
 - II. `./01_submit`
 - III. `./02_check {1st_demo}`
 - 1st_demo deadline: 2024/11/4 (Mon.) 12:00:00
 - 2nd_demo deadline: 2024/11/6 (Wed.) 12:00:00
6. You can key in `./09_clean_up` to clear all log files and dump files in each folder.
7. You need to upload your design and system will name them as **DESIGN_module_iclabxx.v**, **Handshake_syn_iclabxx.v** and **FIFO_syn_iclabxx.v**. (you should check with `./02_check {1st_demo/2nd_demo}`)
8. If the uploaded file violating the naming rule, you will get 5 deduct points.

Waveform Example

1. Asynchronous reset and active-low and reset all output.
2. 6 cycles for input signals



3. 150 cycles for output signals and not required to be continuous

